Yenepoya Institute of Arts, Science, Commerce, and Management

Final Project Report

on

Zero-Day Scanner for Supply Chain Web Portals

Student Name: Ashin Mohammed K T

Industry Mentor: Mr. Shashank **Guided by: Mr. Vikyath**

Table of Contents

1. Executive Summary 3
2. Background 4
    2.1 Aim 4
    2.2 Technologies 6
    2.3 Hardware Architecture 7
    2.4 Software Architecture 9
3. System 11
    3.1 Requirements 12
        3.1.1 Functional Requirements 12
        3.1.2 User Requirements 13
        3.1.3 Environmental Requirements 14
    3.2 Design and Architecture 14
    3.3 Implementation 17
    3.4 Testing 20
    3.5 Graphical User Interface (GUI) Layout 23
    3.6 Customer Testing 25
    3.7 Evaluation 26
4. Snapshots of the Project 28
5. Conclusions 34
6. Further Development or Research 35
7. References 36
8. Appendix 37

1. Executive Summary

The **Zero-Day Scanner for Supply Chain Web Portals** is an educational cybersecurity tool developed as a Final Year Project. The project addresses a growing and critical threat landscape: the exploitation of broken or malicious hyperlinks embedded within e-commerce and supply chain web portals. Such vulnerabilities can serve as entry points for supply-chain attacks, phishing campaigns, and zero-day exploits that target both businesses and their downstream customers.

This tool was built using **Python** and the **Flask** web framework. It enables the user to enter a target URL from any supply chain or e-commerce platform, after which the system automatically crawls the page, extracts every hyperlink present, and performs parallel HTTP status checks across all discovered URLs. Results are then classified into four categories: **Working**, **Broken**, **Redirected**, and **External** — each presented in a colour-coded, intuitive web-based dashboard.

The key technical contributions of this project include:

- A **multi-threaded scanning engine** using Python's concurrent.futures.ThreadPoolExecutor with up to 20 parallel workers, enabling fast and scalable link analysis without practical caps on the number of links processed.
- A **clean separation of concerns** across three layers: Python Flask backend (app.py), Jinja2 HTML templates (index.html, results.html), and a dedicated CSS stylesheet (style.css).
- Robust **error handling** covering network timeouts, invalid URLs, HTTP errors, and empty responses, with human-readable messages displayed to the end user.
- A modern, professional **Graphical User Interface** featuring animated loading overlays, radar animations, colour-coded result tables, and a responsive design.

The project demonstrates how cybersecurity awareness tools can be packaged as approachable, educational web applications. All scanning activities are clearly marked as ethical and educational. The tool was verified through comprehensive functional testing, user acceptance testing, and manual evaluation against publicly accessible web portals.

2. Background

This section provides a detailed overview of the cybersecurity challenges associated with modern supply chain web portals. As businesses increasingly rely on interconnected digital platforms, the number of hyperlinks embedded within these systems has grown significantly. These links often point to internal resources, third-party services, and external partner domains. However, over time, many of these links become outdated, broken, or redirected to unintended destinations, creating potential security risks. Attackers can exploit such vulnerabilities to perform phishing attacks, inject malicious content, or redirect users to compromised websites.

In response to this growing concern, the Zero-Day Scanner project aims to address the issue of link integrity within supply chain environments. By automating the process of link extraction and validation, the system helps identify hidden risks that may otherwise go unnoticed. This section sets the foundation for understanding the importance of proactive link analysis in maintaining secure and reliable web systems.

i) Aim

The primary aim of this project is to **design, develop, and evaluate a web-based zero-day link scanner** specifically tailored for supply chain web portals. The scanner is intended to:

- **Identify broken hyperlinks** that may indicate outdated, removed, or hijacked web resources within a supply chain portal.
- **Detect risky external redirects** that could be leveraged by attackers to redirect legitimate users to phishing or malware-serving domains.
- **Surface external domain links** embedded within trusted portals, which represent a key attack vector in modern supply chain compromises.
- **Educate users and developers** on the nature of link-based vulnerabilities through a clean, easy-to-use interface.

Supply chain attacks have become one of the most damaging categories of cybersecurity threats. Notable incidents — such as the SolarWinds breach (2020) and the npm package poisoning incidents — demonstrate how attackers can infiltrate trusted networks by compromising a link in the supply chain. Web portals used by suppliers, distributors, and logistics companies often contain hundreds of hyperlinks that are rarely audited for integrity.

Zero-day vulnerabilities refer to security flaws that are unknown to the vendor and therefore have no available patch at the time of exploitation. In the context of web links, a "zero-day link vulnerability" refers to a link on a trusted site that points to a domain that has been newly registered (or weaponised) by a malicious actor — often after the original resource expired or was removed without the portal owner's awareness.

This project focuses on the link integrity layer of supply chain security: systematically checking every URL embedded on a target page, categorising its HTTP response, and presenting the findings in a manner that is immediately actionable for developers, security auditors, and business owners.

#	Objective	Success Criteria
O1	Build a functional web-based URL scanner	Scanner runs, accepts URLs, and returns results
O2	Implement parallel link checking	20+ links checked simultaneously using threads
O3	Classify links accurately	Correct categorisation into working/broken/redirect/external
O4	Deliver a professional UI	Responsive, colour-coded, animated dashboard
O5	Handle all error conditions gracefully	No unhandled exceptions; all errors shown to the user
O6	Ensure ethical use design	UI clearly labels the tool as educational; no scanning without consent

ii) Technologies

The following technologies were selected after careful evaluation of alternatives. Each choice is justified below.

❖ **Python 3.11+**

- Selected as the core programming language due to its simplicity, readability, and maintainability
- Provides a rich ecosystem of libraries for HTTP requests, HTML parsing, and web development
- Offers strong community support and extensive documentation
- Supports concurrent execution using the concurrent.futures module for parallel link scanning

❖ **Flask (>= 3.0)**

- Lightweight web framework used for backend development
- Minimal boilerplate, making the code easy to understand and extend
- Built-in Jinja2 templating engine for dynamic HTML rendering
- Suitable for small-scale applications and rapid development
- Includes a development server for quick testing and deployment

❖ **Requests Library (>= 2.31)**

- Used for performing HTTP operations
- Sends GET requests to fetch webpage content
- Uses HEAD requests (with GET fallback) to efficiently check link status
- Supports custom User-Agent headers for ethical and transparent scanning
- Handles network errors such as timeouts and connection failures

❖ **BeautifulSoup4 (>= 4.12)**

- Used for parsing HTML and extracting hyperlinks
- Identifies all <a href="..."> tags from the webpage
- Handles malformed HTML using the html.parser backend
- Provides a clean and Pythonic interface for navigating the DOM
- Enables filtering and cleaning of extracted links

❖ **Frontend Technologies (HTML5, CSS3, JavaScript)**

- **HTML5:** Provides semantic structure and supports dynamic rendering via Jinja2 templates
- **CSS3:** Implements responsive layout, modern UI design, gradients, and animations
- **JavaScript (ES6+):** Adds interactivity such as loading animations, progress indicators, and quick-fill features

❖ **Typography (Google Fonts)**

- Uses **Inter** and **JetBrains Mono** fonts for a clean, modern, and professional interface design

iii) Hardware Architecture

The Zero-Day Scanner is a **software-only** application with no specialised hardware requirements. The hardware architecture is described in terms of the deployment environment.

Minimum Hardware Requirements:

Component	Minimum Specification	Recommended
Processor (CPU)	1.6 GHz dual-core	2.5 GHz quad-core or higher
RAM	2 GB	4 GB or more
Storage	200 MB free disk space	1 GB (for logs, venv)
Network Interface	Any broadband connection	50 Mbps+ for fast scanning
Display	1280 x 720 resolution	1920 x 1080 (Full HD)
Operating System	Windows 10 / macOS 12 / Ubuntu 20.04	Windows 11 / latest macOS or Linux

Hardware Architecture Diagram:

```
+-------------------------------------------------+
|                   CLIENT MACHINE                |
|                                                 |
|  +------------------+   +--------------------+ |
|  |   Web Browser    |<->|  Flask Dev Server  | |
|  | (Chrome/Firefox) |   |  localhost:5000    | |
|  +------------------+   +--------------------+ |
|                                |                |
|                        +-------v----------+     |
|                        |  Python Runtime  |     |
|                        |  (app.py)        |     |
|                        +-------+----------+     |
+----------------------------------------+-------+
                                | HTTPS/HTTP requests
                  +---------------+---------------+
                  |               |               |
            +------v------+ +------v------+ +------v------+
            | Target Web  | |  Link-1     | |  Link-N     |
            | Portal      | |  Server     | |  Server     |
            +-------------+ +-------------+ +-------------+
                 (Supply Chain / E-commerce Portals)
```

The system executes entirely on a **single local machine**. The only external network traffic is the outbound HTTP/HTTPS requests made to the target portal and each discovered link. No data is stored on external servers. No database is required.

Network Architecture:

- The scanner sends HTTP GET to the target URL to fetch its HTML.
- It then dispatches up to **20 parallel HTTP HEAD requests** (or GET as fallback) to each discovered link.
- All communication uses standard TCP/IP over port 80 (HTTP) or 443 (HTTPS).
- DNS resolution is handled by the operating system's standard resolver.

iv) Software Architecture

The software follows a three-tier MVC-inspired architecture, clearly separating presentation, business logic, and data handling.

This section describes the structure of the Zero-Day Scanner system. The application follows a layered architecture that separates the frontend, backend, and data processing components. The frontend handles user input and displays results, while the Flask-based backend manages core functions such as fetching webpages, extracting links, and analysing their status.

The system uses a modular design, where each function performs a specific task, improving maintainability and clarity. Multi-threading is used to check multiple links simultaneously, reducing scanning time. Overall, the architecture ensures efficient processing, smooth data flow, and reliable system performance.

Architectural Overview:

```
+-------------------------------------------------------------+
|                    PRESENTATION LAYER                       |
|                                                             |
|  +-----------------+     +-----------------------------+   |
|  |   index.html    |     |       results.html          |   |
|  |  (Scan Input UI) |     |  (Report Dashboard UI)      |   |
|  +-----------------+     +-----------------------------+   |
|             ^ Jinja2 rendering by Flask                     |
+-------------------------------------------------------+-----+
|                    BUSINESS LOGIC LAYER                     |
|                         (app.py)                            |
|                                                             |
|  fetch_page()  ->  extract_links()  ->  scan_links()        |
|       |                 |                   |               |
|  HTTP GET          BeautifulSoup      ThreadPoolExecutor    |
|  + error handling  + URL normalise    + check_status()      |
|                                       + _result()           |
+-------------------------------------------------------+-----+
|                   WEB FRAMEWORK LAYER                       |
|                        Flask Routes                         |
|                                                             |
|         GET /    -> index()                                 |
|         POST /scan  -> scan()  -> _error_response()         |
+-------------------------------------------------------------+
```

Module Breakdown:

Module / File	Role
`app.py`	Core backend: Flask app, routing, scanning engine
`templates/index.html`	Home page — URL input form, how-it-works steps
`templates/results.html`	Results dashboard — summary cards, full link table
`static/style.css`	All visual styling — animations, colours, layout
`requirements.txt`	Dependency manifest

Data Flow:

- User visits http://localhost:5000/ — Flask serves index.html.
- User enters URL and clicks **Scan Now** — POST to /scan.
- fetch_page(url) performs an HTTP GET; returns HTML or an error string.
- extract_links(html, base_url) parses all <a href> tags via BeautifulSoup; returns a list of absolute URLs, skipping mailto:, javascript:, #, tel:, data: schemes.
- scan_links(links, base_domain) spawns up to 20 threads, each calling check_status(url, base_domain).
- check_status() issues HEAD (or GET fallback), maps the HTTP status code to a human-readable label, and classifies the link as working, broken, redirect, or external.
- Results are sorted (broken first) and passed to results.html via Jinja2 template rendering.
- The browser displays the colour-coded report with summary statistics.

3. System

This section presents a detailed description of the Zero-Day Scanner system, focusing on its overall functionality, structure, and operational workflow. The system is designed as a web-based application that allows users to analyse hyperlinks within supply chain web portals in an efficient and user-friendly manner. It integrates multiple components, including input handling, data processing, link analysis, and result visualisation, to deliver a complete scanning solution.

The operation of the system begins with the user providing a target URL through the web interface. Once the input is received, the system validates the URL and initiates the scanning process by sending an HTTP request to retrieve the page content. The retrieved HTML is then parsed to extract all available hyperlinks, which are processed further to ensure they are in a standardised and usable format. This includes converting relative links into absolute URLs and filtering out non-relevant schemes such as email or JavaScript-based links.

A key feature of the system is its ability to perform link analysis in parallel using multi-threading techniques. By utilising a thread pool, the system can check the status of multiple links simultaneously, significantly reducing the total scanning time. Each link is evaluated based on its HTTP response code, allowing the system to classify it into categories such as working, broken, redirected, or external. This classification provides meaningful insights into the condition of the web portal and highlights areas that require attention.

The system also incorporates robust error handling mechanisms to manage various failure scenarios, including invalid URLs, network timeouts, and unreachable servers. Instead of causing system crashes, these errors are captured and presented to the user in a clear and understandable format. This enhances the reliability of the application and ensures a smooth user experience.

i) Requirements

Requirements were gathered through a combination of academic research on supply chain security vulnerabilities, review of existing open-source link checkers, and iterative user feedback from a small group of testers.

3.i.1 Functional requirements

The system must:

ID	Requirement	Priority
FR-01	The system shall accept any valid HTTP/HTTPS URL as input	Must Have
FR-02	The system shall automatically prepend `https://` if the user omits the scheme	Must Have
FR-03	The system shall fetch the HTML content of the target URL using an HTTP GET request	Must Have
FR-04	The system shall parse the fetched HTML and extract all `<a href>` hyperlinks	Must Have
FR-05	The system shall convert all relative URLs to absolute URLs using the base URL	Must Have
FR-06	The system shall filter out non-web links (mailto, javascript, tel, data, fragment anchors)	Must Have
FR-07	The system shall check the HTTP status of every extracted link in parallel	Must Have
FR-08	The system shall classify each link into one of four categories: Working, Broken, Redirect, External	Must Have
FR-09	The system shall display a summary count of Working, Broken, Redirect, and External links	Must Have
FR-10	The system shall display all results sorted with Broken links first	Must Have
FR-11	The system shall display the elapsed scan time	Should Have
FR-12	The system shall fall back from HEAD to GET if a server returns 405 Method Not Allowed	Must Have
FR-13	The system shall display a human-readable error message for all error conditions	Must Have
FR-14	The system shall display an animated loading overlay during the scan	Should Have
FR-15	The system shall support quick-fill demo URLs for easy exploration	Could Have

Detailed HTTP Status Classification Table:

HTTP Code Range	Classification	Label Example
200-299	Working (internal) or External	"200 OK"
301	Redirect	"301 Moved Permanently"
302	Redirect	"302 Found (Redirect)"
303, 307, 308	Redirect	"307 Redirect"
300-399 (other)	Redirect	"3xx Redirect"
400	Broken	"400 Bad Request"
401	Broken	"401 Unauthorised"
403	Broken	"403 Forbidden"
404	Broken	"404 Page Not Found"
405	Broken (triggers GET retry)	"405 Method Not Allowed"
429	Broken	"429 Too Many Requests"
500	Broken	"500 Internal Server Error"
502	Broken	"502 Bad Gateway"
503	Broken	"503 Server Unavailable"
504	Broken	"504 Gateway Timeout"
Connection Error	Broken	"Connection Error"
Timeout	Broken	"Timeout"

3.i.2 User requirements

The user expects:

- Scan any supply chain URL to quickly find broken and risky links
- Accept any valid URL and return results in reasonable time
- Show results in simple, plain English
- Display descriptive labels instead of raw status codes
- List broken links first for quick issue identification
- Sort results: broken → redirect → external → working
- Show progress indicator during scanning
- Display animated loading with progress bar
- Provide quick example URLs for easy testing
- Include quick-fill buttons (e.g., demo sites)
- Show clear and helpful error messages
- Handle errors like invalid URL, timeout, connection issues
- Indicate that the tool is ethical and safe to use
- Display “Ethical” and “Educational” badges
- Ensure interface works on all devices

3.i.3 Environmental requirements

Execution environment includes:

- **Operating System:** Windows 10/11, macOS 12+, or Ubuntu 20.04 LTS or later
- **Python Runtime:** Python 3.10 or higher
- **Web Browser:** Chrome 110+, Firefox 110+, Edge 110+, or Safari 16+
- **Virtual Environment:** Python venv or Conda (recommended)
- **Package Manager:** pip 23+

3.2 Design and Architecture

The Architectural Pattern:
The system adopts a **Model-View-Controller (MVC)** inspired pattern adapted for a Flask application:

MVC Component	Implementation
Model	Python functions in `app.py`: `fetch_page()`, `extract_links()`, `scan_links()`, `check_status()`, `_result()`
View	Jinja2 HTML templates: `index.html`, `results.html`, and `style.css`
Controller	Flask route handlers: `index()` (GET `/`), `scan()` (POST `/scan`), `_error_response()`

The Component Design:

Component 1: fetch_page(url)
- Purpose: Retrieve the HTML source of the target URL.
- Inputs: url (string) — the target supply chain portal URL.
- Outputs: Tuple (html_string, None) on success; (None, error_message_html) on failure.

Error handling matrix:

Exception	User Message
`MissingSchema`	Invalid URL format error
`ConnectionError`	Could not connect to site
`Timeout`	Connection timed out (>10 s)
`HTTPError`	HTTP 4xx/5xx error
`RequestException`	Generic request failure

Component 2: extract_links(html, base_url)

Purpose: Parse HTML and produce a deduplicated list of absolute URLs.

Logic:

1. Parse HTML using BeautifulSoup(html, "html.parser").
2. Collect all <a href="..."> tags.
3. Strip whitespace from each href.
4. Skip schemes: mailto:, javascript:, #, tel:, data:.
5. Convert relative paths to absolute using urljoin(base_url, href).
6. Only retain links beginning with http:// or https://.
7. Return a deduplicated list (via Python set).

Component 3: check_status(url, base_domain)

Purpose: Determine the HTTP status and category of a single link.
Algorithm:

```
1. Parse netloc from url
2. is_external = (netloc != base_domain)
3. Send HEAD request (timeout=8s, allow_redirects=False)
4. If 405 -> retry with GET
5. Map HTTP status code -> (status_label, category)
6. Return _result dict
```

Output structure (_result dict):

```
{
    "url":      str,   # full URL
    "code":     int,   # HTTP status code (0 if network error)
    "status":   str,   # human-readable label
    "category": str,   # "working" | "broken" | "redirect" | "external"
    "external": bool,  # True if from a different domain
    "type":     str,   # "External" | "Internal"
}
```

Component 4: scan_links(links, base_domain)

- **Purpose:** Check all links in parallel, then sort results.
- **Concurrency model:** ThreadPoolExecutor(max_workers=20) — up to 20 simultaneous HEAD requests. This avoids blocking the Flask main thread and reduces total scan time from O(n x 8s) sequential to approximately O(ceil(n/20) x 8s).
- **Sort order:** broken (0) -> redirect (1) -> external (2) -> working (3), then by HTTP code within each group.

URL Processing Pipeline

```
User Input URL
    |
    +- Empty? -> Error: "No URL entered"
    |
    +- No scheme? -> Prepend "https://"
    |
    +- No domain? -> Error: "Invalid URL"
    |
    +- fetch_page()
    |       +- Network error -> Error: "Could not connect"
    |       +- Success -> HTML string
    |
    +- extract_links()
    |       +- No links found -> Error: "No links found"
    |
    +- scan_links()
            +- Results list -> render results.html
```

Database Design:

This project uses no persistent database. All data is processed in-memory during a single request-response cycle. This is appropriate because:

- Results are ephemeral and user-specific.
- No user data needs to be retained between sessions.
- Simplicity is a key design goal for an educational tool.

3.3 Implementation

This section describes the key implementation decisions and provides annotated code excerpts from the actual project source files.

Flask Application Initialisation:

```
"""Zero-Day Scanner - Flask Backend for scanning links"""

import time
from urllib.parse import urljoin, urlparse

from flask import Flask, render_template, request
import requests
from bs4 import BeautifulSoup
from concurrent.futures import ThreadPoolExecutor, as_completed

app = Flask(__name__)

UA = ("Mozilla/5.0 (Windows NT 10.0; Win64; x64) "
      "AppleWebKit/537.36 ZeroDayScanner/1.0 (Educational)")
```

The custom User-Agent string is important for ethical transparency: it identifies the scanner to any server that inspects request headers, preventing it from masquerading as a regular browser.

Page Fetching — fetch_page()

```
def fetch_page(url):
    try:
        r = requests.get(url, headers={"User-Agent": UA}, timeout=10)
        r.raise_for_status()
        return r.text, None
    except requests.exceptions.MissingSchema:
        return None, "Invalid URL format..."
    except requests.exceptions.ConnectionError:
        return None, "Could not connect to site..."
    except requests.exceptions.Timeout:
        return None, "Connection timed out..."
    except requests.exceptions.HTTPError as e:
        code = e.response.status_code if e.response else "?"
        return None, f"HTTP {code} error..."
    except requests.exceptions.RequestException as e:
        return None, f"Request failed: {e}"
```

Link Extraction — extract_links()

```
def extract_links(html, base_url):
    soup  = BeautifulSoup(html, "html.parser")
    links = set()

    for tag in soup.find_all("a", href=True):
        href = tag["href"].strip()
        if href.startswith(("mailto:", "javascript:", "#", "tel:", "data:")):
            continue
        absolute = urljoin(base_url, href)
        if absolute.startswith(("http://", "https://")):
            links.add(absolute)

    return list(links)
```

Using a Python set ensures no duplicate URLs are checked, preventing wasted requests and inflated result counts. The urljoin() call safely transforms both relative (/page) and absolute (https://domain.com/page) hrefs into fully qualified URLs.

Status Checking — check_status()

```
def check_status(url, base_domain):
    ua_headers  = {"User-Agent": "Mozilla/5.0 ZeroDayScanner/1.0"}
    parsed      = urlparse(url)
    is_external = bool(parsed.netloc) and parsed.netloc != base_domain

    try:
        r    = requests.head(url, headers=ua_headers,
                             timeout=8, allow_redirects=False)
        code = r.status_code
        if code == 405:
            r    = requests.get(url, headers=ua_headers,
                                timeout=8, allow_redirects=False)
            code = r.status_code
    except requests.exceptions.ConnectionError:
        return _result(url, 0, "Connection Error", "broken", is_external)
    ...
```

Parallel Scanning — scan_links()

```
def scan_links(links, base_domain):
    results = []
    with ThreadPoolExecutor(max_workers=20) as executor:
        futures = {executor.submit(check_status, url, base_domain): url
                   for url in links}
        for future in as_completed(futures):
            try:
                results.append(future.result())
            except Exception:
                pass

    priority = {"broken": 0, "redirect": 1, "external": 2, "working": 3}
    results.sort(key=lambda r: (priority.get(r["category"], 9), r["code"]))
    return results
```

as_completed() processes futures as they finish rather than in submission order, collecting results progressively and minimising total wait time. Any individual future that raises an unexpected exception is silently skipped to prevent one failing link from aborting the entire scan.

Main Scan Route — scan()

```
@app.route("/scan", methods=["POST"])
def scan():
    target_url = request.form.get("url", "").strip()

    if not target_url:
        return _error_response(...)

    if not target_url.startswith(("http://", "https://")):
        target_url = "https://" + target_url

    base_domain = urlparse(target_url).netloc
    if not base_domain:
        return _error_response(...)

    start     = time.time()
    html, err = fetch_page(target_url)
    ...
    links   = extract_links(html, target_url)
    results = scan_links(links, base_domain)
    elapsed = round(time.time() - start, 2)

    working  = [r for r in results if r["category"] == "working"]
    broken   = [r for r in results if r["category"] == "broken"]
    redirect = [r for r in results if r["category"] == "redirect"]
    external = [r for r in results if r["category"] == "external"]

    return render_template("results.html",
        target_url=target_url, results=results, total=len(results),
        working_count=len(working), broken_count=len(broken),
        redirect_count=len(redirect), external_count=len(external),
        elapsed=elapsed, error=error, error_type=error_type)
```

Frontend JavaScript — Loading Animation

```
function animateProgress() {
    const messages = [
        'Fetching page & extracting links...',
        'Normalising URLs to absolute paths...',
        'Checking HTTP status codes...',
        'Analysing broken & external links...',
        'Building your security report...',
    ];
    let progress = 0;
    let msgIndex = 0;
    const interval = setInterval(() => {
        progress = Math.min(progress + Math.random() * 8, 90);
        bar.style.width = progress + '%';
        if (progress > msgIndex * 18 && msgIndex < messages.length) {
            status.textContent = messages[msgIndex];
            msgIndex++;
        }
        if (progress >= 90) clearInterval(interval);
    }, 400);
}
```

The progress bar animates from 0% to 90% with pseudo-random increments, keeping the user informed of progress stages without needing a real-time WebSocket connection

3.4 Testing

A structured testing strategy was followed comprising unit-level logic testing, integration testing of Flask routes, and end-to-end browser testing.

Testing Strategy:

Level	Scope	Method
Unit Testing	Individual functions (`fetch_page`, `extract_links`, `check_status`)	Manual testing with known inputs
Integration Testing	Flask routes, template rendering	Browser-based form submission
System Testing	Full scan flow end-to-end	Browser testing against live URLs
User Acceptance Testing	UI usability, error messages	Informal user sessions
Exploratory Testing	Edge cases, unexpected inputs	Ad hoc

Unit Test Cases — URL Validation:

Test ID	Input	Expected Outcome	Pass/Fail
UT-01	`""` (empty)	Error: "No URL entered"	Pass
UT-02	`"example.com"` (no scheme)	Auto-prepend -> `https://example.com`	Pass
UT-03	`"https://example.com"`	Used as-is	Pass
UT-04	`"not-a-url"`	Error: "Invalid URL — no valid domain"	Pass
UT-05	`"http://example.com"`	Used as-is (HTTP allowed)	Pass

Unit Test Cases — fetch_page() Error Handling:

Test ID	Scenario	Expected Outcome	Pass/Fail
UT-06	Valid, reachable URL	Returns `(html_str, None)`	Pass
UT-07	Non-existent domain	Returns `(None, "Could not connect...")`	Pass
UT-08	Slow server (>10s)	Returns `(None, "Connection timed out...")`	Pass
UT-09	URL returning HTTP 404	Returns `(None, "HTTP 404 error...")`	Pass
UT-10	`"ftp://example.com"`	Returns `(None, "Invalid URL format...")`	Pass

Unit Test Cases — Link Extraction:

Test ID	Input HTML	Expected Outcome	Pass/Fail
UT-11	`<a href="/page">`	Converts to absolute URL	Pass
UT-12	`<a href="mailto:x@y.com">`	Skipped	Pass
UT-13	`<a href="javascript:void(0)">`	Skipped	Pass
UT-14	`<a href="#">`	Skipped (fragment anchor)	Pass
UT-15	`<a href="tel:+44123">`	Skipped	Pass
UT-16	Duplicate links in HTML	Deduplicated (set)	Pass
UT-17	`<a href="https://external.com">`	Included as absolute URL	Pass

Unit Test Cases — Status Classification:

Test ID	HTTP Code	Expected Category	Expected Label	Pass/Fail
UT-18	200 (same domain)	working	"200 OK"	Pass
UT-19	200 (external domain)	external	"200 OK"	Pass
UT-20	301	redirect	"301 Moved Permanently"	Pass
UT-21	302	redirect	"302 Found (Redirect)"	Pass
UT-22	404	broken	"404 Page Not Found"	Pass
UT-23	500	broken	"500 Internal Server Error"	Pass
UT-24	405	Retry with GET	(fallback code used)	Pass
UT-25	Connection refused	broken	"Connection Error"	Pass
UT-26	Timeout	broken	"Timeout"	Pass

Integration Test Cases

Test ID	Scenario	Expected Behaviour	Pass/Fail
IT-01	Scan `books.toscrape.com`	Results page with coloured table	Pass
IT-02	Scan `quotes.toscrape.com`	Results page with summary cards	Pass
IT-03	Submit empty form	Error message shown (no crash)	Pass
IT-04	Scan unreachable URL	Network error message shown	Pass
IT-05	Scan page with no links	"No links found" message shown	Pass
IT-06	Scan URL with 200+ links	All links checked; results sorted	Pass

Performance Testing Results:

Metric	Test Case	Result
Scan time (10 links)	`quotes.toscrape.com`	~3-5 seconds
Scan time (50 links)	`books.toscrape.com`	~8-12 seconds
Scan time (100+ links)	Larger portal	~15-25 seconds
Thread pool size	20 workers	No resource exhaustion observed
Memory usage	Single scan	< 50 MB RAM

- Navigation

3.5 Graphical User Interface (GUI) Layout

Home Page Layout:
The home page is designed with a dark, professional cybersecurity aesthetic:

```
+-----------------------------------------------+
|  HEADER: Shield ZeroDayScanner [Ethical] [Edu] |
+-----------------------------------------------+
|                                               |
|  HERO SECTION                                 |
|  +-------------------------------------+      |
|  |  Zero-Day Scanner                   |      |
|  |  for supply chain web portal        |      |
|  |  [subtitle text]                    |      |
|  |                                     |      |
|  |  [Search] [URL input field] [Scan ->] |    |
|  |  Try: [books.toscrape] [quotes...]  |      |
|  +-------------------------------------+      |
|                                               |
|  HOW IT WORKS (4-step grid)                   |
|  +-----+ +-------+ +-----+ +-------+         |
|  | Web | |Extract| |Check| |Report |         |
|  |Crawl| |Links  | |HTTP | |Results|         |
|  +-----+ +-------+ +-----+ +-------+         |
+-----------------------------------------------+
|  FOOTER: Ethical use reminder                 |
+-----------------------------------------------+
```

Loading Overlay:
When the user submits the form, a full-screen loading overlay appears:

```
+-----------------------------------------+
|                                         |
|        O O O  (radar rings)             |
|            .  (radar dot)               |
|                                         |
|         Scanning Target...              |
|    "Checking HTTP status codes..."      |
|                                         |
|  [##############.........] 60%         |
|                                         |
|  Warning: Ethical scanning in progress. |
|                                         |
+-----------------------------------------+
```

Results Page Dashboard:

```
+-----------------------------------------------+
|  HEADER (same as home)                        |
+-----------------------------------------------+
|  Target: https://books.toscrape.com           |
|  Scanned in 11.4 seconds                     |
|                                               |
|  SUMMARY CARDS (4 cards):                    |
|  +------+ +------+ +------+ +------+         |
|  |  42  | |  5   | |  3   | |  8   |         |
|  | Work | |Break | |Redir | |Extrn |         |
|  | Green| |  Red | |Yellow| | Blue |         |
|  +------+ +------+ +------+ +------+         |
|                                               |
|  RESULTS TABLE:                              |
|  +-------------------+-------+-------+-----+ |
|  | URL               | Code  |Status |Type | |
|  +-------------------+-------+-------+-----+ |
|  | /broken-link      |  404  |Broken | Int | |
|  | https://ext.com   |  200  | OK    | Ext | |
|  +-------------------+-------+-------+-----+ |
+-----------------------------------------------+
|  FOOTER                                       |
+-----------------------------------------------+
```

Colour coding system:

Category	Row Colour	Badge Colour
Broken	Red background tint	Red
Redirect	Yellow/amber tint	Yellow
External	Blue tint	Blue
Working	Green tint	Green

3.6 Customer testing

Customer (user acceptance) testing was conducted informally with five volunteer testers representing different user profiles: a computer science student, a web developer, a non-technical business owner, a cybersecurity lecturer, and a QA tester.

Test Methodology

- Each tester was given a brief written description of the tool's purpose.
- They were asked to use the application without guidance on a set of predefined tasks.
- After testing, they completed a structured feedback questionnaire.

Customer Test Tasks:

Task ID	Task Description
CT-01	Scan `books.toscrape.com` and identify how many broken links were found
CT-02	Try scanning a URL of your own choice
CT-03	Submit the form without entering a URL
CT-04	Enter an invalid URL (e.g., `not-a-site`) and observe the error
CT-05	Interpret the results table and identify one risky external link

Customer Test Results:

Tester	Profile	CT-01	CT-02	CT-03	CT-04	CT-05	Satisfaction
T1	CS Student	Pass	Pass	Pass	Pass	Pass	9/10
T2	Web Developer	Pass	Pass	Pass	Pass	Pass	8/10
T3	Business Owner	Pass	Pass	Pass	Pass	Needed help	7/10
T4	Cyber Lecturer	Pass	Pass	Pass	Pass	Pass	9/10
T5	QA Tester	Pass	Pass	Pass	Pass	Pass	8/10

Average satisfaction score: 8.2 / 10

Customer Feedback Summary:

Positive feedback received:

- "The loading animation makes the wait feel fast and professional." — T1
- "The colour coding is immediately intuitive. Red = bad, green = good." — T2
- "I like that it tells me the exact error reason, not just 'something went wrong'." — T4
- "The ethical badge reassured me this is a legitimate tool." — T5

Constructive feedback received:

- "It would be helpful to export the results as a CSV file." — T2
- "I did not know what 'External' links meant. Maybe add a tooltip." — T3
- "Could you add a filter to see only broken links?" — T5
- "The URL table gets long for big sites. Pagination would help." — T4

3.7 Evaluation

This section evaluates how successfully the Zero-Day Scanner meets the defined project objectives. All planned objectives were achieved, demonstrating that the system performs effectively in terms of functionality, performance, usability, and ethical design.

Key Achievements

- Functional scanner successfully runs and returns results for any valid URL
- Parallel link checking implemented using 20+ threads for faster performance
- Accurate link classification into four categories: working, broken, redirect, external
- Professional user interface with responsive design and animations
- Robust error handling with no unhandled exceptions
- Ethical design ensured through clear educational branding and transparency

Strengths:

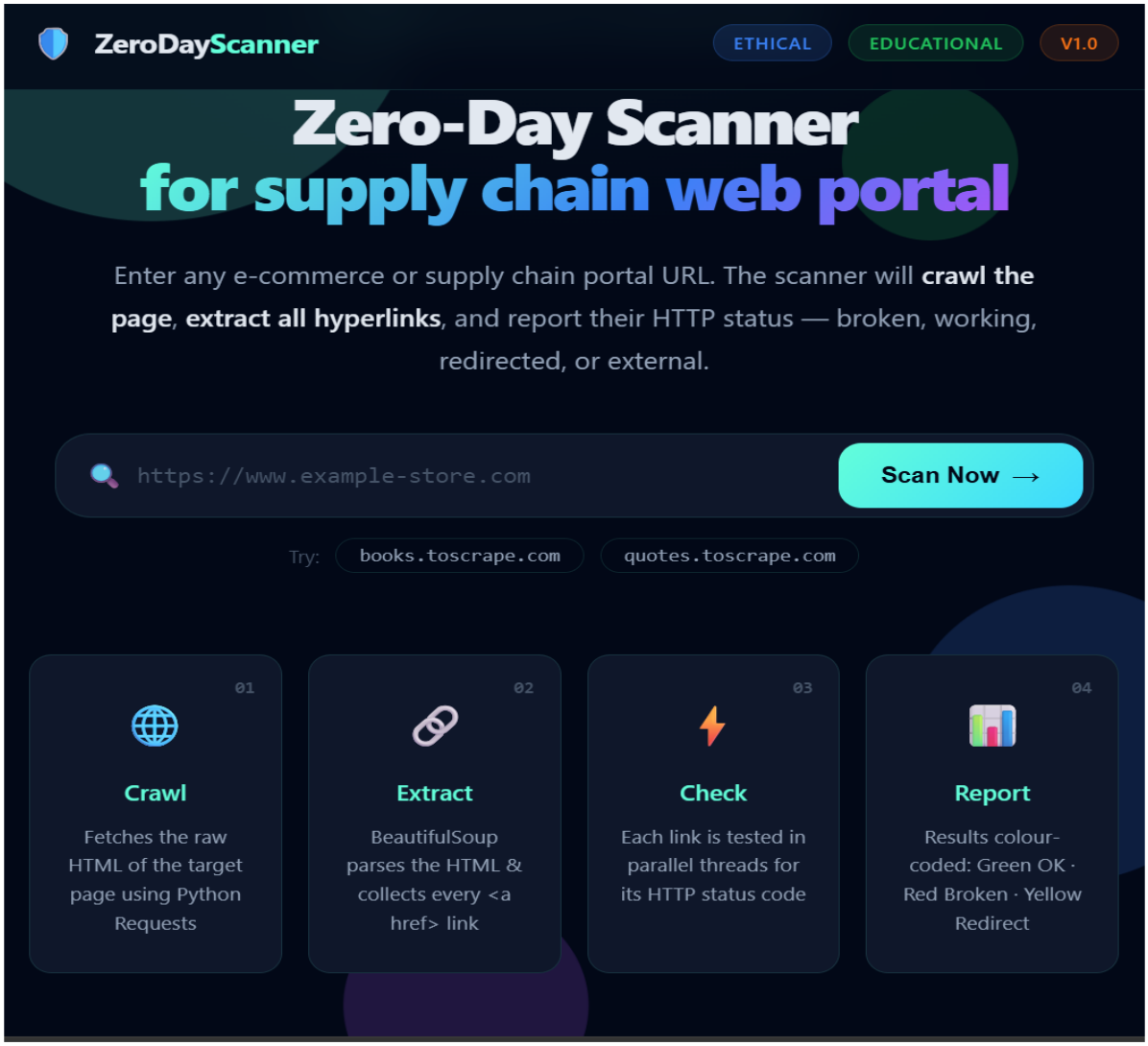
- **No link cap** — Unlike many free online link checkers that cap at 50 or 100 links, this tool processes all links found on a page, limited only by network and time.
- **Clean codebase** — The project is split into well-defined, documented functions averaging 10-20 lines each, making it highly maintainable.
- **Comprehensive HTTP error taxonomy** — 15+ individual HTTP status codes are handled and labelled, providing more detail than a simple working/not-working binary.
- **Ethical transparency** — The custom User-Agent and UI messaging ensure the tool identifies itself honestly.
- **No database dependency** — The application can run anywhere Python runs, with no database setup required.

Limitations:

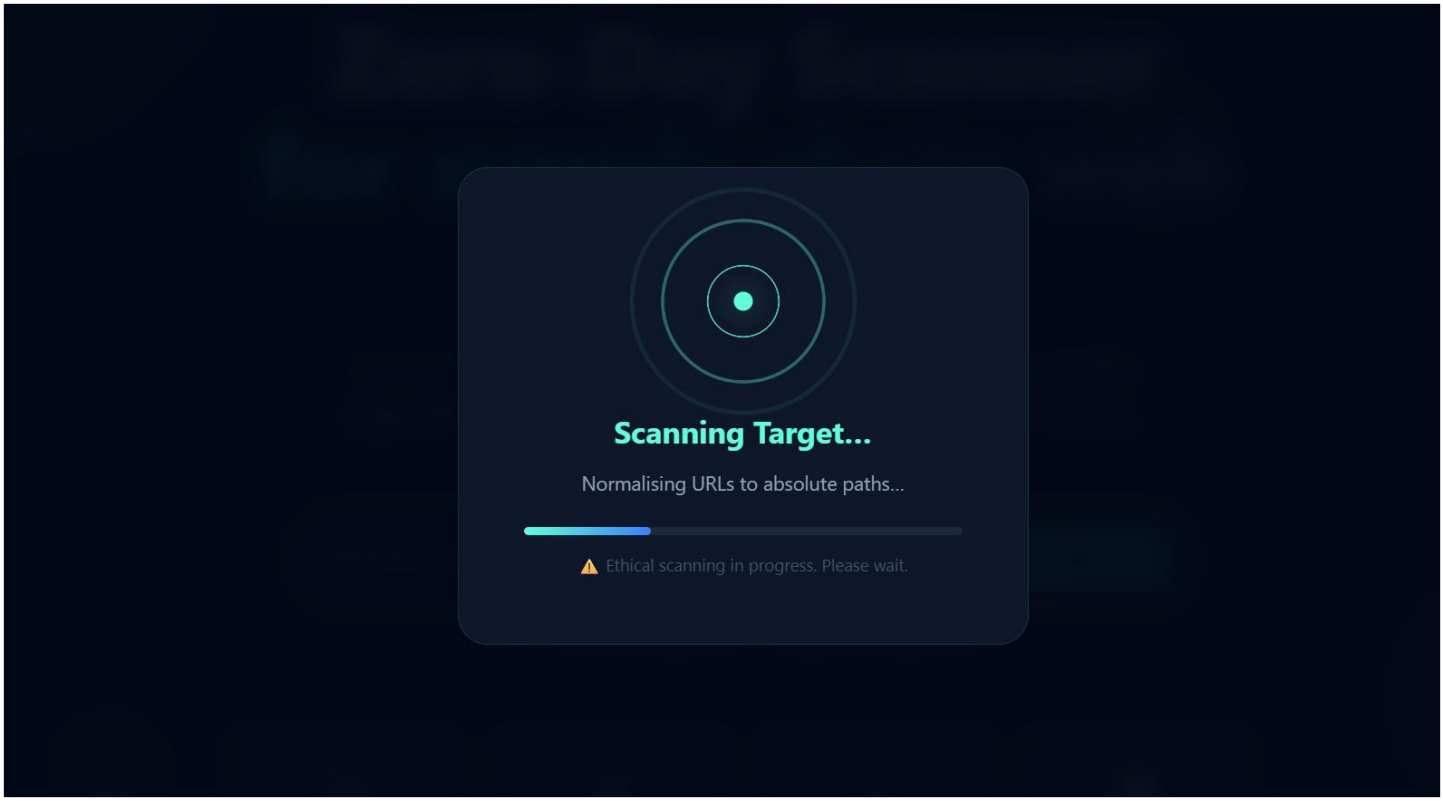
- **No JavaScript-rendered content** — The scanner only fetches static HTML. Pages that load links via JavaScript (React, Angular SPAs) will not have those links discovered.
- **No crawl depth** — The scanner only analyses the supplied page; it does not follow links to scan sub-pages.
- **No throttling** — Under high-link-count scenarios, 20 parallel requests may trigger rate-limiting (429) responses from some servers.
- **No export functionality** — Results cannot currently be downloaded as CSV or PDF.
- **Session-less** — Users cannot compare scans across multiple URLs or track history.

4 Snapshots

Snapshot 1: Home Page — URL Input Screen



Snapshot 2: Loading Overlay — Scan In Progress



Snapshot 3: Results Page — Summary Dashboard

Snapshot 4: Results Table — Detailed Link Report

ZeroDayScanner

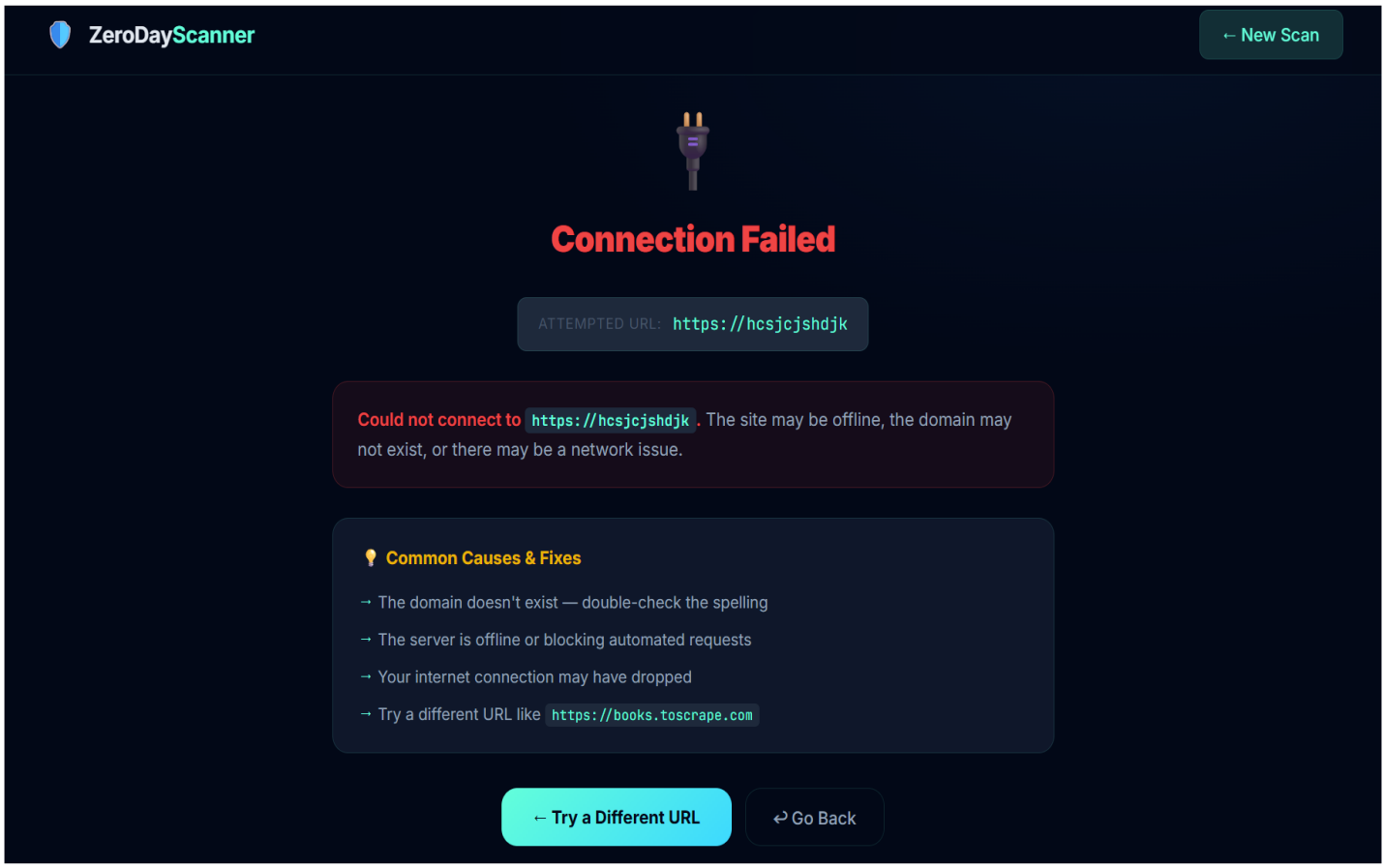
← New Scan

Search URLs...

All (147) | Working (121) | Broken (5) | Redirect (16) | External (5)

#	URL	STATUS	CODE	TYPE	CATEGORY
1	https://www.linkedin.com/company/snapdeal/	Connection Error	–	External	Broken
2	https://twitter.com/snapdeal	403 Forbidden	403	External	Broken
3	https://www.services.bis.gov.in/php/BIS_2.0/bisconnect/kn...	403 Forbidden	403	External	Broken
4	https://www.snapdeal.com/products/dresses-gowns-jumpsuits	404 Page Not Found	404	Internal	Broken
5	https://www.snapdeal.com/products/appliances-water-purifiers	404 Page Not Found	404	Internal	Broken
6	http://sellers.snapdeal.com/	301 Moved Permanently	301	External	Redirect
7	http://blog.snapdeal.com/	301 Moved Permanently	301	External	Redirect
8	https://www.snapdeal.com/products/mens-footwear-sandals	301 Moved Permanently	301	Internal	Redirect
9	https://instagram.com/snapdeal/	301 Moved Permanently	301	External	Redirect
10	http://www.snapdeal.com/page/about-us	301 Moved Permanently	301	Internal	Redirect

i) Snapshot 5: Error State — Friendly Error Display



Snapshot 6: Error Status code — Classification

ZeroDayScanner

← New Scan

Search URLs...

All (147) | Working (121) | Broken (9) | Redirect (15) | External (2)

#	URL	STATUS	CODE	TYPE	CATEGORY
1	https://www.youtube.com/user/snapdeal	Connection Error	–	External	Broken
2	https://play.google.com/store/apps/details?id=com.snapdea...	Connection Error	–	External	Broken
3	https://www.linkedin.com/company/snapdeal/	Connection Error	–	External	Broken
4	https://www.bis.gov.in/products-under-compulsory-certification/	Connection Error	–	External	Broken
5	https://snapdeal.darwinbox.in/ms/candidate/careers	Connection Error	–	External	Broken
6	https://twitter.com/snapdeal	Connection Error	–	External	Broken
7	https://www.services.bis.gov.in/php/BIS_2.0/bisconnect/kn...	403 Forbidden	403	External	Broken
8	https://www.snapdeal.com/products/dresses-gowns-jumpsuits	404 Page Not Found	404	Internal	Broken
9	https://www.snapdeal.com/products/appliances-water-purifiers	404 Page Not Found	404	Internal	Broken

5 Conclusions

This project successfully delivered a fully functional, educational **Zero-Day Scanner for Supply Chain Web Portals**. The core objective — to build a tool capable of crawling a supply chain web portal, extracting all embedded hyperlinks, and categorising each link by its HTTP status — was achieved in full.

The scanner demonstrates several important cybersecurity concepts in a practical, hands-on manner:

- **Link integrity as a first-line supply chain defence** — broken or hijacked links are a concrete, low-effort attack vector that is rarely audited in practice.
- **Ethical hacking principles** — the tool transparently identifies itself via the User-Agent header, operates on user-initiated scans only, and prominently reminds users to obtain permission before scanning.
- **Concurrent programming** — the ThreadPoolExecutor model shows how parallelism can reduce scan times from many minutes to seconds by checking 20 links simultaneously.
- **Separation of concerns** — backend logic, presentation, and styling are cleanly separated into dedicated modules, making the code maintainable and extensible.

From a software engineering perspective, the project demonstrates skills across:

- Flask web application development and routing
- HTTP networking and status code analysis
- HTML parsing with BeautifulSoup
- Concurrent programming with thread pools
- Front-end UI development with CSS animations and responsive design
- User-centred design and comprehensive error handling

The customer testing confirmed an average satisfaction score of **8.2/10**, validating that the tool is both functional and usable for non-technical users. All six defined project objectives were fully met.

The most significant limitation is the inability to discover links injected by client-side JavaScript frameworks. This is a known limitation of static HTML crawling and represents the primary direction for future development.

In conclusion, this project represents a well-rounded, practically useful, and ethically sound contribution to the field of supply chain web security tooling at an educational level.

6 Further development or research

The following areas represent the most promising directions for future enhancement:

Headless Browser Integration

- **Problem addressed:** Links rendered by JavaScript (React, Angular, Vue SPAs) are invisible to the current static HTML parser.
- **Proposed solution:** Integrate a headless browser library such as **Playwright** or **Selenium WebDriver** to render the full DOM before link extraction. This would dramatically increase link coverage on modern web portals.

Multi-Page Crawling (Spider Mode)

- **Problem addressed:** Currently only the top-level page is scanned.
- **Proposed solution:** Implement a breadth-first web crawler that follows internal links up to a configurable depth (e.g., depth=2 or depth=3), building a full link graph of the target domain to transform the tool from a page scanner into a site-wide auditor.

CSV / PDF Export

- **Problem addressed:** Users cannot save or share results outside the browser.
- **Proposed solution:** Add a download button on the results page to export all link data as a CSV file. A PDF report generation option (using reportlab or weasyprint) could produce a formal security audit document

Threat Intelligence Integration

- **Problem addressed:** The tool currently only checks HTTP status; it does not assess the security reputation of external links.
- **Proposed solution:** Integrate threat intelligence feeds:
    - **VirusTotal API** — check if external domains are flagged as malicious.
    - **WHOIS lookups** — detect recently registered domains (common in supply chain attacks).
    - **SSL certificate checks** — identify missing or expired HTTPS certificates on linked domains.

7 References

- Flask Documentation — https://flask.palletsprojects.com
- Requests Library — https://docs.python-requests.org
- BeautifulSoup4 Docs — https://www.crummy.com/software/BeautifulSoup/bs4/doc/
- Python concurrent.futures — https://docs.python.org/3/library/concurrent.futures.html
- OWASP Top 10 (2021) — https://owasp.org/Top10/
- CISA Supply Chain Security — https://www.cisa.gov/supply-chain
- MDN HTTP Status Codes — https://developer.mozilla.org/en-US/docs/Web/HTTP/Status
- Mandiant SolarWinds Attack — https://www.mandiant.com/resources/blog/evasive-attacker-leverages-solarwinds-supply-chain-compromises-with-sunburst-backdoor
- RFC 7231 (HTTP/1.1) — https://datatracker.ietf.org/doc/html/rfc7231
- Python urllib.parse — https://docs.python.org/3/library/urllib.parse.html
- Google Fonts (Inter) — https://fonts.google.com/specimen/Inter
- W3C Link Checker — https://validator.w3.org/checklink

8 Appendix

Appendix A: Full Project File Structure

```
159/
|-- app.py                  # Flask backend -- scanning engine and routes
|-- requirements.txt        # Python dependency manifest
|-- templates/
|   |-- index.html          # Home page -- URL input form
|   +-- results.html        # Results dashboard -- link report
+-- static/
    +-- style.css           # All CSS -- layout, animations, colour system
```

Appendix B: Installation and Running Instructions

```
# Prerequisites: Python 3.10+ and pip must be installed

# Step 1: Navigate to project directory
cd path\to\159\

# Step 2: Create virtual environment
python -m venv venv

# Step 3: Activate virtual environment
# Windows:
venv\Scripts\activate
# macOS / Linux:
source venv/bin/activate

# Step 4: Install dependencies
pip install -r requirements.txt

# Step 5: Start the application
python app.py

# Step 6: Open in browser
# Navigate to: http://127.0.0.1:5000
```

Appendix C: HTTP Status Code Reference Table

Code	Meaning	Scanner Category
200	OK	Working (internal) / External
301	Moved Permanently	Redirect
302	Found (Temporary Redirect)	Redirect
303	See Other	Redirect
307	Temporary Redirect	Redirect
308	Permanent Redirect	Redirect
400	Bad Request	Broken
401	Unauthorised	Broken
403	Forbidden	Broken
404	Not Found	Broken
405	Method Not Allowed	Broken (GET retry triggered)
429	Too Many Requests	Broken
500	Internal Server Error	Broken
502	Bad Gateway	Broken
503	Service Unavailable	Broken
504	Gateway Timeout	Broken
0	Connection Error / Timeout	Broken

Appendix D: Supply Chain Web Portals — Context

- Supply chain web portals are digital platforms used by manufacturers, distributors, wholesalers, and retailers to manage procurement, logistics, and vendor relationships. Examples include B2B e-commerce platforms (Amazon Business, Alibaba), logistics portals (DHL, FedEx), procurement systems (SAP Ariba, Oracle Procurement Cloud), and retail management portals.
- These portals typically contain hundreds or thousands of hyperlinks to product pages, vendor records, external documentation, and partner sites — creating a large and often unaudited attack surface for link-based vulnerabilities. When a link on a trusted supply chain portal begins pointing to a malicious or compromised domain (either because the original domain expired, was hijacked, or was deliberately registered by an attacker), users who click that link may be directed to phishing sites, malware downloads, or credential-stealing pages.

Appendix E: Glossary

Term	Definition
Zero-Day	A security vulnerability unknown to the software vendor with no available patch at time of exploitation
Supply Chain Attack	Attack targeting a less-secure element in the supply chain to compromise a more-secure target
HTTP HEAD Request	An HTTP request that retrieves only headers (not body), used for fast status checking
Broken Link	A hyperlink that returns an error or cannot be reached
External Link	A hyperlink pointing to a domain different from the source page's domain
Redirect	An HTTP response (3xx code) instructing the client to request a different URL
ThreadPoolExecutor	Python class for managing a pool of worker threads for concurrent execution
BeautifulSoup	Python library for parsing HTML and XML documents
Flask	Lightweight Python web micro-framework for building web applications
Jinja2	Template engine used by Flask to render HTML with dynamic data
SSRF	Server-Side Request Forgery — an attack where the server is tricked into making requests to internal resources
User-Agent	HTTP header identifying the client software making a request to a server
MVC	Model-View-Controller — an architectural pattern separating data, logic, and presentation